COMPLETE BUILD LAB

Git + Maven + Jenkins + Docker

Practical Exam Guide

Step-by-step procedure with code, commands, explanations, where to use each item, when to use it, verification steps, and troubleshooting.

Goal: Build a Java application, package it with Maven, store it in GitHub, and use Jenkins to build and test a Docker image automatically.

Follow the steps in order. Replace placeholder GitHub URLs and usernames with your own.

1. What the practical is asking you to do

Create a small Java program, use Maven to compile/package it, use Git to track the source and GitHub to host it, then configure Jenkins to run the build stages and Docker to package and execute the application.

Tool	Role	Why / when it is used
Git	Version control	Track changes and create commits.
GitHub	Remote repository	Host code so Jenkins can fetch it.
Maven	Java build tool	Compile, test if configured, and package a JAR.
Jenkins	Automation server	Run build steps automatically through a pipeline.
Docker	Container platform	Package the JAR with a Java runtime and run it consistently.

Pipeline flow

```
App.java -> pom.xml / Maven -> target/mock-build-lab-1.0.jar
    -> Git commit -> GitHub repository
    -> Jenkinsfile -> Checkout -> Maven Build -> Docker Build -> Docker Test
    -> Output: Complete Build Lab Successful!
```

Jenkins is running in a Linux Docker container in this practice setup, so the pipeline uses `sh` commands, not Windows-only `bat` commands.

2. Prerequisites and folder structure

PowerShell is the Windows host. Jenkins runs inside a Linux container, so check tools in the environment that will execute each command.

```
# Windows PowerShell
java -version
git --version
docker --version
mvn -version

# Inside the Jenkins container
docker exec jenkins java -version
docker exec jenkins mvn -version
docker exec jenkins docker --version
docker exec jenkins docker ps
```

In this practice setup, Maven was manually installed inside the running Jenkins container. If that container is recreated, the manual installation may be lost. An exam environment may already have Maven configured or may require a Jenkins tool/custom image.

Expected structure

```
D:\mock-build-lab\
|-- pom.xml
|-- Dockerfile
|-- Jenkinsfile
|-- .gitignore (optional)
`-- src\main\java\com\example\App.java
```

Maven expects `pom.xml` at the project root and Java files under `src/main/java` by default. Keep Dockerfile and Jenkinsfile at the repository root unless you configure different paths.

3. Complete practical procedure

Step 1 — Create folders

Where: PowerShell. **When:** At the start of a new project.

```
D:
mkdir mock-build-lab
cd D:\mock-build-lab
mkdir src\main\java\com\example
```

Purpose: creates the standard Maven Java source path. `cd` changes the current directory.

Step 2 — Create App.java

Where: `D:\mock-build-lab\src\main\java\com\example\App.java`. **When:** Before compiling.

```
notepad src\main\java\com\example\App.java
```

If asked, choose Yes to create the file. Paste and save:

```
package com.example;

public class App {
    public static void main(String[] args) {
        System.out.println("Complete Build Lab Successful!");
    }
}
```

Why: this is the Java source. `package com.example` matches the folder path; `main` is the entry point; `println` provides a visible result for testing.

Step 3 — Create pom.xml

Where: `D:\mock-build-lab\pom.xml`, at project root. **When:** Before Maven commands.

```
notepad pom.xml
```

Paste and save as `pom.xml`, not `pom.xml.txt`:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>mock-build-lab</artifactId>
    <version>1.0</version>
    <properties>
        <maven.compiler.release>21</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.13.0</version>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-jar-plugin</artifactId>
                <version>3.4.2</version>
                <configuration>
                    <archive><manifest><mainClass>com.example.App</mainClass></manifest></archive>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Why: `groupId`, `artifactId`, and `version` identify the project and determine the JAR name. Java release 21 sets the target. The compiler plugin compiles source; the JAR plugin adds `Main-Class` to the manifest so `java -jar` can run the program.

Step 4 — Build with Maven

Where: PowerShell in `D:\mock-build-lab`, where `pom.xml` exists. **When:** After creating/changing source or build configuration.

```
mvn clean package
```

Expected: `BUILD SUCCESS`. The JAR is `target\mock-build-lab-1.0.jar`. `clean` removes old build output; `package` runs the Maven lifecycle through packaging.

Verify the JAR:

```
java -jar target\mock-build-lab-1.0.jar
```

Expected: `Complete Build Lab Successful!`. If `no main manifest attribute` appears, check the Maven JAR plugin's `mainClass` and rebuild.

Step 5 — Create Dockerfile

Where: `D:\mock-build-lab\Dockerfile`, project root. **When:** After the JAR exists.

```
notepad Dockerfile
```

Paste and save:

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/mock-build-lab-1.0.jar app.jar
CMD ["java", "-jar", "app.jar"]
```

Why: `FROM` chooses the Java 21 runtime image; `WORKDIR` sets the directory inside the image; `COPY` adds the JAR; `CMD` specifies the command when a container starts.

Step 6 — Build and run Docker

Where: PowerShell at the project root. **When:** After Maven succeeds.

```
docker build -t mock-build-lab:1.0 .
docker run --rm mock-build-lab:1.0
```

Expected output: `Complete Build Lab Successful!`. `-t` names/tags the image; `.` means current directory is the build context; `--rm` removes the stopped test container.

Step 7 — Initialize Git and commit

Where: PowerShell at project root. **When:** After creating the files.

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
```

Why: `git init` creates a local repository; `git add .` stages changes; `git commit` records a snapshot; `git branch -M main` names the branch `main`. If Git asks for identity:

```
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

Recommended `.gitignore` contents:

```
target/
```

This keeps generated build output out of source control.

Step 8 — Create GitHub repository and push

Create a public GitHub repository named `mock-build-lab`. For this procedure, leave README initialization unchecked.

```
git remote add origin https://github.com/YOUR-USERNAME/mock-build-lab.git
git push -u origin main
```

Replace `YOUR-USERNAME` with your real GitHub username. `origin` is the conventional remote name; `push` uploads commits; `-u` configures upstream tracking.

If `origin` already exists:

```
git remote set-url origin https://github.com/YOUR-USERNAME/mock-build-lab.git
git push -u origin main
```

Step 9 — Create Jenkinsfile

Where: `D:\mock-build-lab\Jenkinsfile`, project root. **When:** Before configuring the Jenkins job.

```
notepad Jenkinsfile
```

Paste the pipeline, replacing the repository URL:

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/YOUR-USERNAME/mock-build-lab.git'
            }
        }
```

```
        stage('Maven Build') {
            steps {
                sh 'mvn clean package'
            }
        }
        stage('Docker Build') {
            steps {
                sh 'docker build -t mock-build-lab:1.0 .'
            }
        }
        stage('Docker Test') {
            steps {
                sh 'docker run --rm mock-build-lab:1.0'
            }
        }
    }

    post {
        success {
            echo 'Complete Build Lab finished successfully!'
        }
        failure {
            echo 'Build failed. Check Console Output.'
        }
    }
}
```

Why: `pipeline` declares a declarative pipeline; `agent any` asks Jenkins to use an available agent; `stages` divides work into named steps; `git` checks out code; `sh` runs Linux shell commands; `post` reports success or failure. The agent needs Java, Maven, Docker CLI, and Docker daemon access.

Important: Linux Jenkins uses `sh`. Windows Jenkins agents typically use `bat`.

Step 10 — Push Jenkinsfile

```
git add Jenkinsfile
git commit -m "Add Jenkins pipeline"
git push
```

Why: Jenkins loads the pipeline from GitHub, so the Jenkinsfile must be committed and pushed.

Step 11 — Configure Jenkins

Open `http://localhost:8080` and do the following:

• Click **New Item**.

• Name the job `Mock-Build-Pipeline`.

• Choose **Pipeline** and click **OK**.

• Under Pipeline, set Definition to **Pipeline script from SCM**.

• Set SCM to **Git**.

• Enter the GitHub repository URL.

• Set Branch Specifier to ***/main**.

• Set Script Path to **Jenkinsfile**.

• Click **Save**, then **Build Now**.

Why: Pipeline script from SCM tells Jenkins to load the Jenkinsfile from the repository. Branch Specifier selects the branch; Script Path identifies the pipeline file.

Step 12 — Verify success

Open the build number, then Console Output and/or the stage view. Expected stages: Checkout, Maven Build, Docker Build, Docker Test. A successful build ends with `Finished: SUCCESS`, and Docker Test prints `Complete Build Lab Successful!`.

If submission screenshots are required, capture the green stage view, console output, GitHub repository files, and Docker run output.

4. Troubleshooting common errors

`mvn` not found

Maven is missing from the environment running the command. Check Windows and Jenkins separately; configure Maven for the Jenkins agent.

`Batch scripts can only be run on Windows nodes`

A Linux Jenkins agent is running a `bat` step. Replace it with `sh`.

`docker: command not found`

Docker CLI is missing in the Jenkins agent/container.

Cannot connect to Docker daemon / permission denied

Docker CLI exists, but the agent cannot access the Docker socket/daemon or lacks permission.

Docker `COPY ...jar: not found`

The JAR was not built, the name/path is wrong, or Docker build was run from the wrong directory.

`no main manifest attribute`

Configure the JAR plugin with `mainClass`, then rebuild.

`remote origin already exists`

Use `git remote set-url origin ` instead of adding another remote.

Jenkins cannot find Jenkinsfile

Check the branch, that Jenkinsfile was committed/pushed, and Script Path is exactly `Jenkinsfile`.

GitHub authentication failure

Check repository URL and visibility. Private repositories need valid credentials configured in Jenkins.

Base image pull failure

Check network access and the image/tag `eclipse-temurin:21-jre`.

5. Quick revision table

Item	What / why	Where / when
App.java	Java source and testable output	src/main/java; create first
pom.xml	Maven metadata and build plugins	Project root; before Maven
mvn clean package	Compile and package JAR	Project root; after source changes
Dockerfile	Instructions to build an image	Project root; after JAR exists
docker build	Build an image	Project root; after Maven
docker run	Run a container	After image build
Git commit/push	Record and upload source	After creating/changing files
Jenkinsfile	Pipeline as code	Repository root
Pipeline from SCM	Load Jenkinsfile from GitHub	Jenkins job configuration
sh	Run Linux shell commands	Linux Jenkins agent
post	Handle success/failure actions	Inside Jenkinsfile

6. Quick viva questions and answers

Q. What is Git?

A. A distributed version-control system that records source-code changes.

Q. What is GitHub?

A. A hosted Git repository service from which Jenkins can fetch code.

Q. What is Maven?

A. A Java build and dependency-management tool.

Q. What does `mvn clean package` do?

A. `clean` removes previous output; `package` runs the lifecycle through packaging and produces a JAR.

Q. What is a JAR?

A. A Java archive containing compiled classes/resources; it can be executable when its manifest names a main class.

Q. Docker image vs container?

A. An image is the packaged template; a container is an instance created from that image.

Q. What is Jenkins?

A. An automation server that runs jobs and pipelines.

Q. What is a Jenkinsfile?

A. A file containing pipeline-as-code instructions, often stored with the source.

Q. What is CI?

A. Continuous Integration: frequently integrating changes and automatically building/testing them to find issues early.

Q. Why use stages?

A. They organize the pipeline and make it easier to see which step failed.

Q. Why use `sh` here?

A. The Jenkins agent runs Linux.

Q. Why must the JAR exist before Docker build?

A. The Dockerfile's COPY instruction adds that JAR into the image.

7. Last-minute exam checklist

- ■ Create the Maven folder structure.
- ■ Write App.java with a main method.
- ■ Create pom.xml and explain the Main-Class setting.
- ■ Run `mvn clean package` and locate the JAR.
- ■ Create Dockerfile and explain each instruction.
- ■ Build and run the Docker image.
- ■ Initialize Git, commit, add GitHub remote, and push.
- ■ Create and commit Jenkinsfile.
- ■ Configure Pipeline script from SCM.
- ■ Identify all four Jenkins stages.
- ■ Troubleshoot Maven missing, wrong shell command, missing JAR, and Docker daemon access.

Final memory aid: Code → Maven → GitHub → Jenkins → Docker → Verify output.